CONTEXTUAL LEARNING FOR INTERRUPTION TOLERANCE AND MULTITASKING

Thomas E. Portegys, portegys@gmail.com , ORCID 0000-0003-0087-6363

Dialectek, DeKalb, Illinois USA

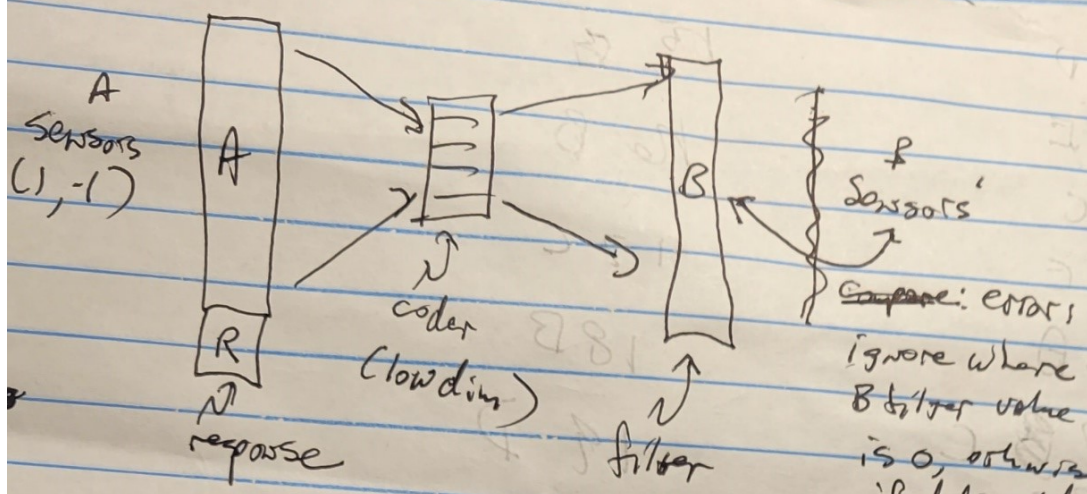
ABSTRACT

This project is an effort to combine the Morphognosis and Mona neural network models into a comprehensive model for learning and behavior called Mandala. Mona features a contextual causation learning with goal-directed motivation. Morphognosis features contextual multilayer perceptron (MLP) learning. Mandala achieves this by externally accumulating tiers of temporal states that are fed into an MLP at each time step. Natural environments abound in event streams that require interruption tolerance and multitasking. Mandala affords multitasking as it is robust in the presence of intervening events representing overlaid causation streams, a capability that conventional recurrent artificial neural networks (RNNs) struggle with. In addition, externally accumulating temporal information discretely labels hierarchical cause-and-effect relationships that can be used for augmented processing. In the case of Mona, channeling motivation through the network for the purpose of goal-seeking requires this feature.

Keywords: Mona, Morphognosis, causation learning, multitasking, artificial neural network, machine learning.

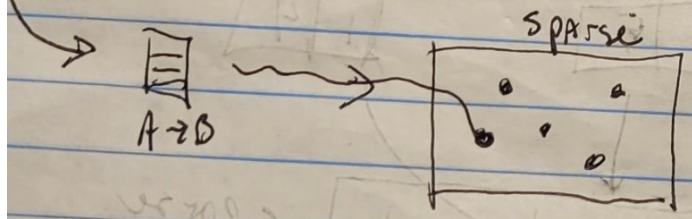
Notes:

Note: sparse example: A: (3,5) with two dimensions after dimension reduction from input, B: (1,4). Plot points into a 10x10 space. Repeat process going up each level. So a dimension value pivots to a range of dimension in the higher space (similar to DB pivot).



Coder stored as A→B in MONA network, which uses
ensemblement & motivation
eval of A→B is f(proximity of coder to prototype, using clustering w/relative
error of B)
closer weighs error more)
→ is worse eval

Coder at lower level up sampled using quantized vector va
Input to entire NN is entire stack of levels



Train from env & let natural causations arise
hopefully delays in changes at higher levels
results in longer time spanning causations

How to achieve variable causation latencies at levels. Causations in higher levels will have greater time spans than lower levels. One method for this would be to vary the proximity parameter for the code centroid spaces. Centroids that can be close together will capture small changes; centroid that are farther apart will each be mapped to by a wider variation of codes and thus require more changes to map to a different centroid.

Addressing the A->B, X->Y co-occurrence event, in this case the code may lie between the A->B and the X->Y centroids. So why not process both of them? Neither will show up as strongly as if they were the sole causation event, but they should prove significant also.

2/13/2022

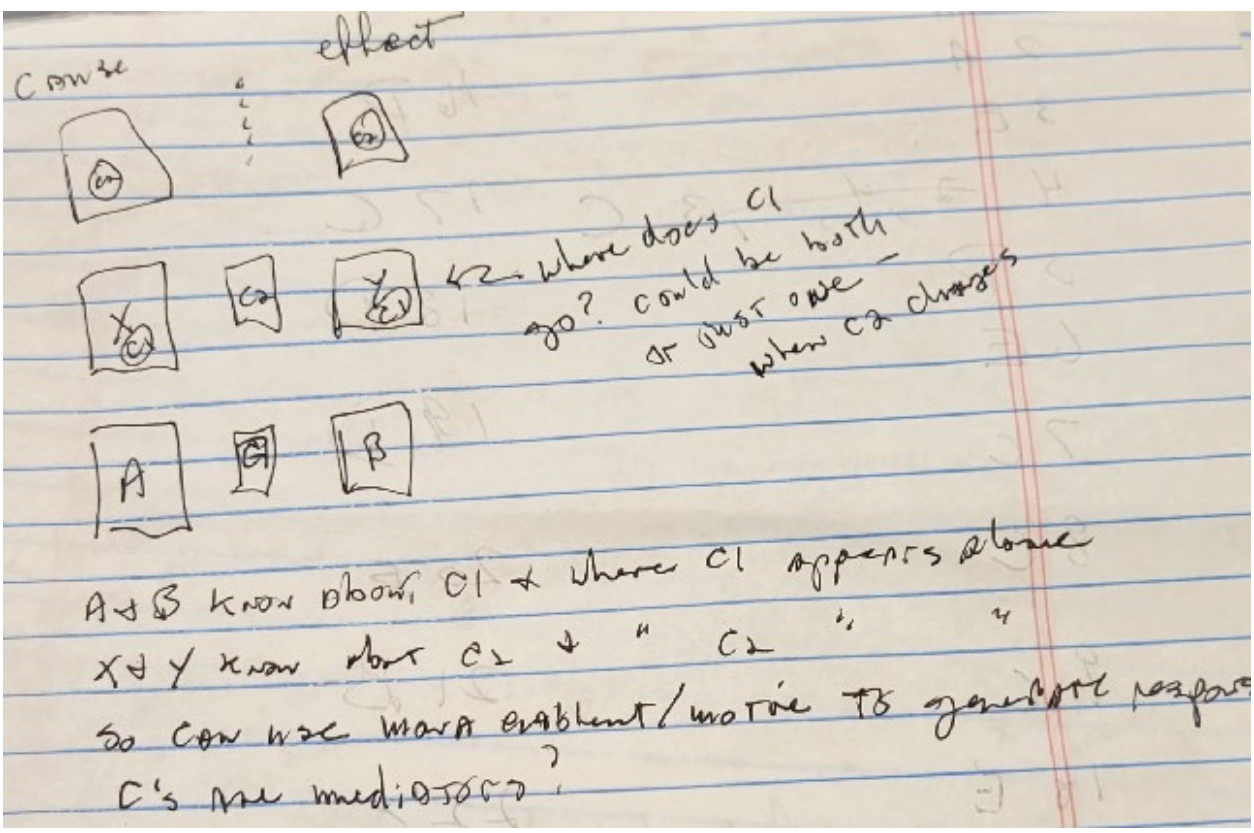
Let there be two computations:

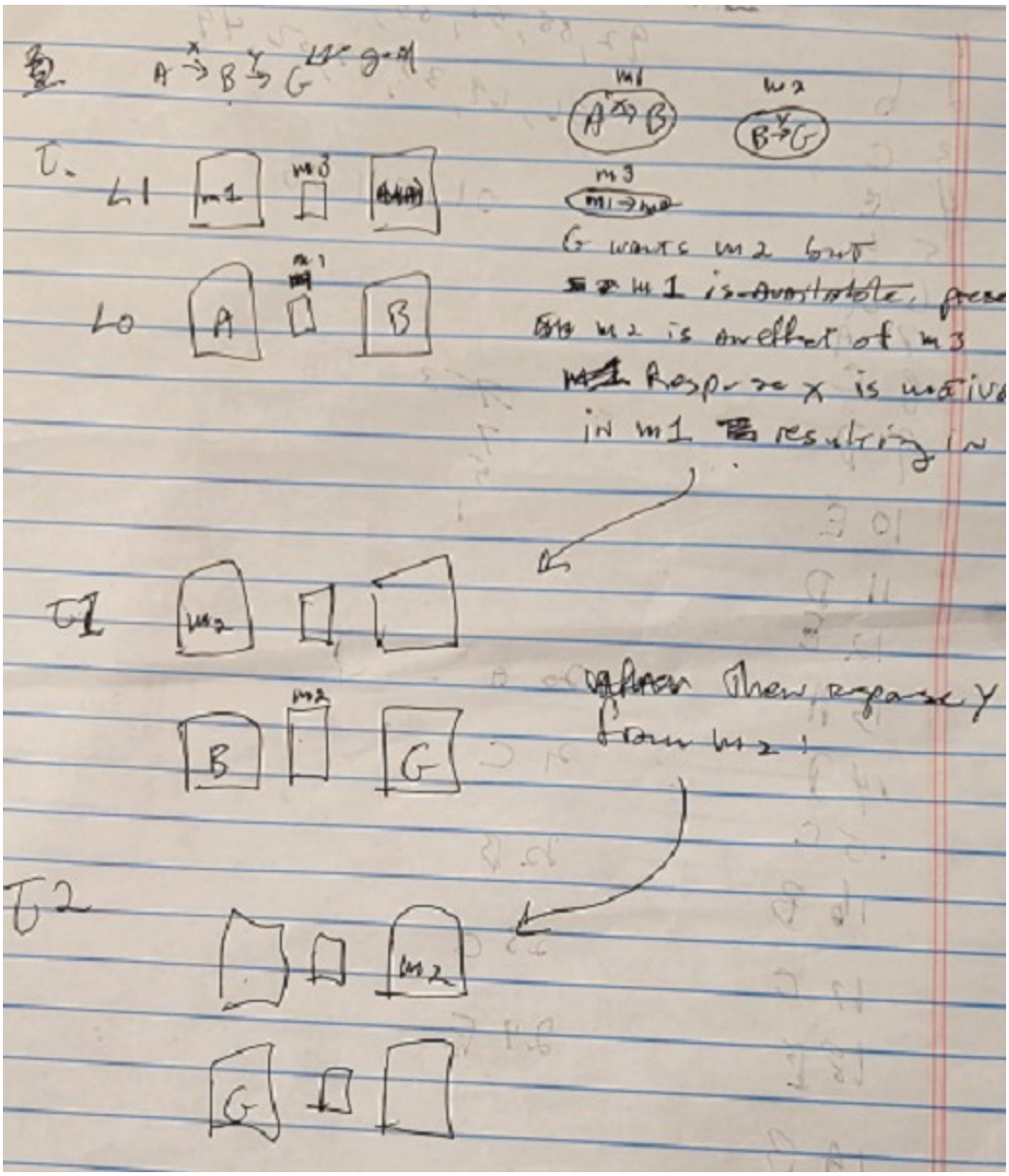
Sensors at all levels -> response (not including need, as this is a model-based system as opposed to a model-free system aka Q-learning).

Per level + response -> effect (this will create the code to be incorporated at higher level)

Can this scheme dispense with the Mona network and centroids? (no)

2/14/2022





Ref:

Exploratory State Representation Learning

Astrid Merckling*, Nicolas Perrin-Gilbert, Alex Coninx and Stéphane Doncieux

Model-based scheme to explore and map env followed by RL on learned space.

Note: look up Reber learning comparison between humans and machines. Notes how humans appear to learn it piecemeal like Mandala.

Instead of using centroids, might use an RBF network to classify against prototype vectors.

Instead of generating random grammars, generate them according to rules:

1. Breadth: e.g. A, AB, ABC
2. Depth: A=A1 A2 A3
3. Repetition: AA, ABAB, ABCABC

At the edge of the tree, terminals are produced.

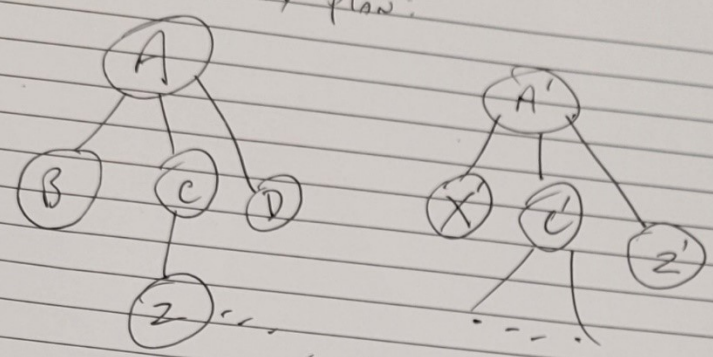
This will provide more reliable performance measurements.

Mandala prototyping with grammar. Instead of encoding causations, sum the occurrences, so if a and b occur in the lower level in the level's time duration, the one-hot representations of a and b are summed.

What sort of statistical analysis can be done on the mutual info/Baysian probs of the grammar that help explain performances?

Spruce up TDNN as a prototype test. Try testing entire length, accumulated values, copying values into multiple layers, and subsumption. These things helped for the maze project.

Modularity plan:

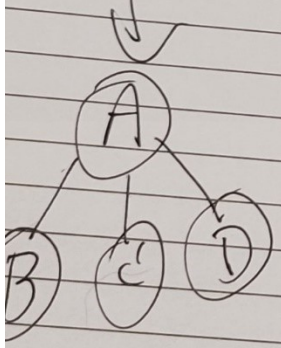


(1) learn both independently

(2) swap ~~out~~ corresponding nodes from A' to A e.g. C' → C

(3) ~~Another~~ challenge is ~~to~~ how well can navigate using new node even though modular has been learned

(4) next: how quickly can hence this



(5) point being how modular learning works

Adapt "horizon" w/ normalization

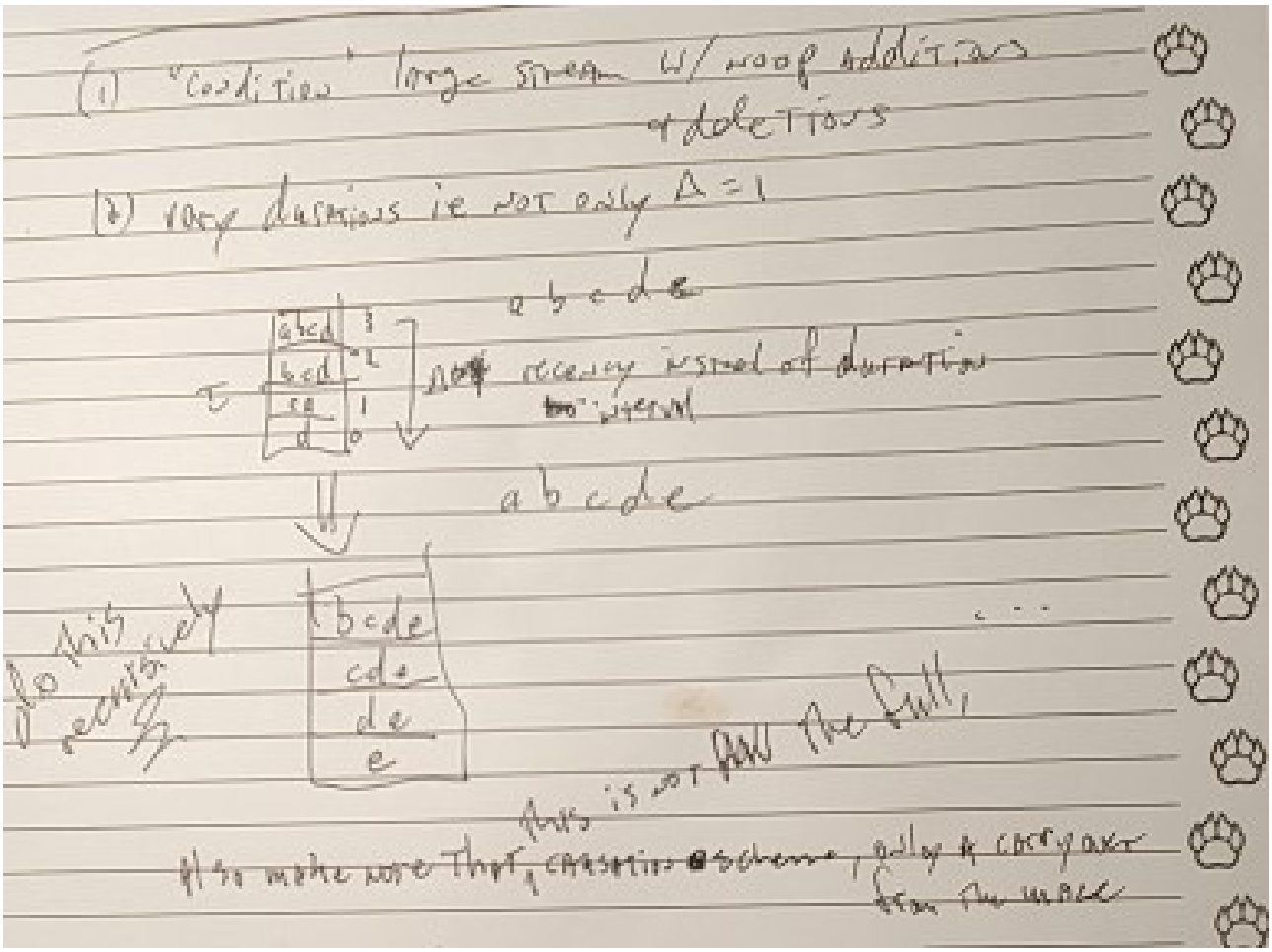
so if ~~see~~ a, b, c:

abc
~~a~~bc
c

A simplification:

Start -> Goal. In between are a number of subgoals in sequence. These can be learned separately/modularly. String some of them together and learn the whole stream. Then life intervenes and adds and subtracts subtasks from the stream. Test and measure this.

Let the Start -> Goal be one monolithic learning stream. Then the task is to complete it given deletion and insertions of other modules.



Hierarchical Reinforcement Learning: Assignment of Behaviours to Subpolicies by Self-Organization

Wilco Moerman
Cognitive Artificial Intelligence, Utrecht University
wilcom@phil.uu.nl, wilco.moerman@gmail.com

https://www.geeksforgeeks.org/deep-learning/sparse-autoencoders-in-deep-learning/

Remember to check EverNote and Melendey for refs.

Mona:

A pair of cooperating nest-building and foraging birds.

See: http://tom.portegys.com/research.html#nestingbirds

Morphognosis:

Honey bees forage for flower nectar cooperatively.

See: http://tom.portegys.com/research.html#honey_bees

1/13/2026

The design has context tier 0 features being assigned to sensory events that are significant to learning and recognizing pattern hierarchies. This assignment, as well as the higher tier context processing, takes place outside of the neural network. So the system as a whole consists of both the neural network and the process that accompanies the network. In a real-world scenario, significant sensory events might be determined by attention and/or emotional impact.

2/19/2026

Other recurrent architectures with external memory. Also rely on RNN/LSTM though. Not as integrated as Mandala.

AI Overview

Recurrent Neural Networks (RNNs) with external temporal memory are

advanced architectures designed to overcome the limitations of standard RNNs (like LSTMs or GRUs) in memorizing long-term dependencies. While standard RNNs rely on internal hidden states that often suffer from vanishing gradients over long sequences, these models integrate a separate, addressable memory bank to store and retrieve past information over extended time horizons.

Core Components and Functionality

Controller (RNN): A standard neural network (e.g., LSTM) that processes inputs and decides, via a controller, what information to write to or read from the external memory.

External Memory Bank: A matrix or tensor (often addressable) that acts as a long-term storage, allowing the model to "recall" information from far in the past.

Addressing Mechanism: Techniques like content-based addressing or location-based addressing allow the model to select specific memory locations to read from or write to.

Key Architectures

Neural Turing Machines (NTMs): A foundational model that couples a neural network controller to an external memory bank, capable of learning to manipulate and manage the memory to solve tasks like sorting or copying.

Memory Networks (MemNN/MemN2N): These models use external memory to store input data, which can be queried later to aid in tasks like question answering.

External Addressable Long-Term and Working Memory (EALWM)-Augmented RNNs: An advanced structure that splits external memory into distinct long-term and working memory parts, improving the ability to learn long-term dependencies without gradient issues.

Differentiable Neural Computers (DNCs): A further extension of NTMs that provides better dynamic memory allocation and temporal memory linkage.

Advantages

Long-Term Dependencies: Capable of capturing relationships over thousands of steps, whereas LSTMs may fail.

Improved Memory Capacity: The memory capacity is decoupled from the number of hidden units in the controller, allowing for much larger, efficient storage.

Task Versatility: Highly effective for complex tasks such as algorithmic learning, language modeling, question answering, and reasoning, which require tracking historical data.

Applications and Performance

These models are specifically used in scenarios demanding significant context, such as:

Language Understanding: Associating tags to words in long text sequences.

Algorithm Learning: Tasks like copying, sorting, and associative recall.

Question Answering: Recalling information from stories or documents.

They are often compared against standard LSTM models, showing superior performance in scenarios where memory length requirement exceeds the inherent capacity of the LSTM's gating mechanism.

https://arxiv.org/abs/1506.00195

Recurrent Neural Networks with External Memory for Language Understanding

This model has a long-term memory that is addressable from the RNN and which feeds into the hidden memory.

2/20/2026

Tried Claude, Gemini, ChatGPT on test_0 sequences.

Given these sequences:

0,1,2,3

4,5,2,6

Predict the next number in these sequences:

8,8,7,0

7,11,4

7,4,5,11,11,8,10,10,10,11,2

0,1,9,11,10,2

7,9,11,9,7,8,4,5,7,10,10,8,7,2

7,0,1,11,10,7,7,10,2

4,5,10,8,8,2

11,8,0,1,8,10,2

11,7,7,11,11,0,1,11,7,11,11,9,11,10,7,2

10,8,9,7,4,5,9,8,8,10,11,11,7,9,9,11,2

Note that 4,5 causes 2,3 and 0,1 causes 2,6

Claude predicted the first two simple sequences, But failed on the sequence that depended on recognizing the earlier transitions determine the ending transitions.

ChatGPT did not correctly predict any of the sequences.

2/24/2026

Note for paper: possible to learn out-of-order streams, but not done here. Reference my paper on conjunctions.